evolvyn

PLATFORM ARCHITECTURE / EXTENSIBILITY

Extensible agent platform

Registries, guardrails and gateways

How Evolvyn adds connections and capabilities through defined contracts while preserving authorization, bounded execution and human decisions.

MCP REGISTRY

Extend connections and discovery

TOOL REGISTRY

Extend callable business capabilities

GOVERNANCE / GUARDRAILS

Extend validated policy and checks

GATEWAYS

Extend model and tool adapters

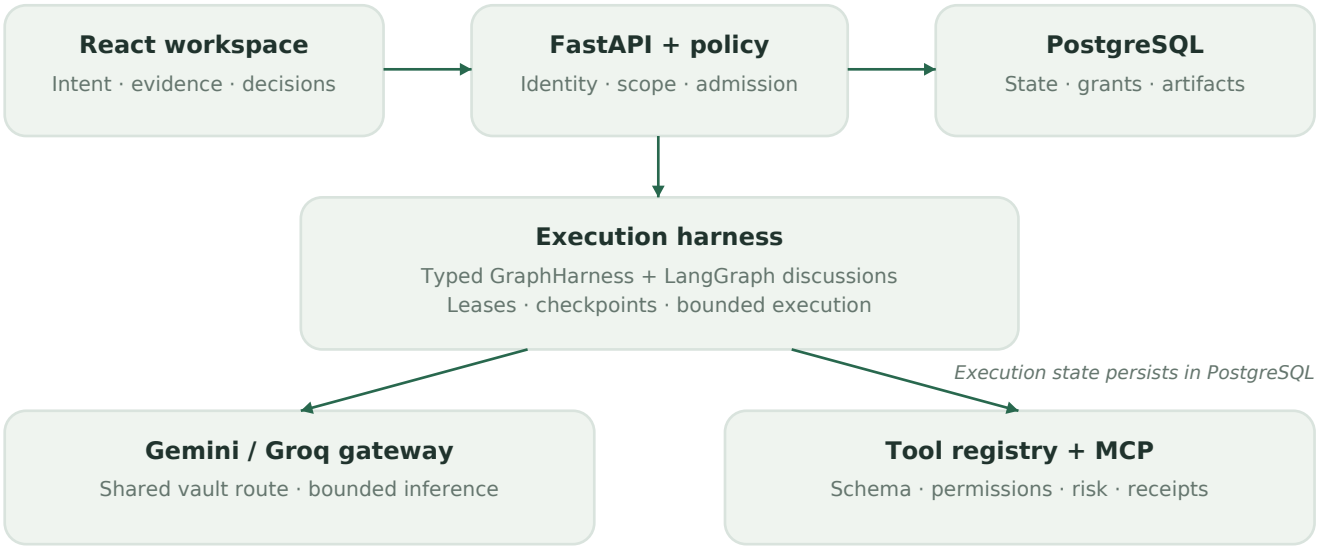
Standalone engineering brief · 25 October 2026 / Source **8ac1e56**. Supported registration and configuration are distinguished from engineering extensions. Live provider readiness requires separate acceptance.

10 pages · Dedicated component sections · Extensibility matrix · Source references

01 / ARCHITECTURE AT A GLANCE

Extensible components, explicit boundaries

Evolvyn combines a bounded execution harness with extensible registries, deterministic governance and application-level gateways. New connections and capabilities enter through defined contracts. The platform controls identity, scope, budgets, credentials, approval and recorded results around those capabilities.



Four extension surfaces

- **MCP registry:** add compatible, authorized servers and discover their tools.
- **Tool registry:** expose existing implementations or discovered MCP tools through stable names, schemas, capabilities and business entities.
- **Governance and guardrails:** configure policy for the workspace; implement additional checks in reviewed backend code when existing controls are insufficient.
- **Gateways:** extend model-provider and tool-adapter implementations through their shared execution boundaries.

The harness is also extensible: validated graphs can compose registered tools, agents, conditions, approvals and results. New role definitions can reference permitted tools. New behavior still consumes budgets and remains subject to policy.

Extensibility has two levels: supported configuration/registration at runtime, and engineering changes that require implementation, tests and deployment. The following pages distinguish them. Extensible does not mean that arbitrary code, arbitrary model providers or arbitrary external actions can be enabled through a dropdown.

Sources: [S01], [S03], [S04], [S05], [S06], [S07], [S09].

02 / MCP REGISTRY

Extend the platform's connection inventory

The MCP registry stores organization-scoped server records: name, transport/profile, endpoint, credential reference, risk, timeout, enabled state, health and discovered tools. It describes where a capability is available and how to establish a session. It does not authorize every exposed operation.

Runtime extension: register a compatible server

1. An owner/admin chooses a supported bundled profile or a public HTTPS Streamable HTTP endpoint.
2. For a remote endpoint, the hostname is explicitly approved. Unsafe/private destinations, credential-bearing URLs and nonstandard HTTPS ports are rejected.
3. An enabled MCP credential is referenced through the vault, or an allowed private environment binding is used. The raw token is not placed in the endpoint or tool arguments.
4. A health probe initializes the MCP session and discovers tool definitions. The registry validates discovered schemas and records the actual result.
5. Selected discovered tools can then be registered for use through the Tool Registry.

Engineering extension: add a native profile

A new bundled stdio profile requires a reviewed module, a permitted profile mapping/schema, controlled process configuration and tests. A new provider's OAuth flow also requires an implemented authorization lifecycle, scoped storage and callback/expiry/revocation handling. Registering a server is not a generic OAuth implementation.

Checks remain in force

The transport revalidates approved destinations and uses bounded calls. Invocation verifies current remote tool existence, arguments and any returned structured payload covered by an advertised output schema. Tool output is size-bounded and remains untrusted evidence.

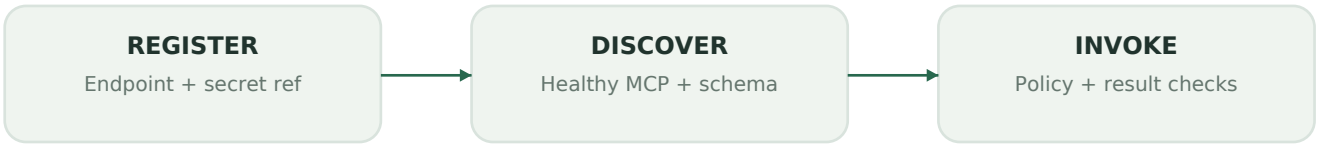
The standalone MCP starter illustrates the extension path with calculate_conversion and compare_conversion_rates. Its authenticated endpoint can be registered without changing Evolvyn's business logic. A server record, a healthy session and a registered usable tool are separate milestones.

Sources: [S03], [S04], [S10], [S11].

03 / TOOL REGISTRY

Extend what agents and workflows can call

The Tool Registry resolves a stable tool name to an implementation contract. Metadata includes description, adapter, input schema, risk, permission, timeout, capability labels and business-entity labels. Built-in handlers also have typed output validation; remote tools use discovered protocol contracts where supplied.



Non-read tool → exact-argument approval before execution

Runtime extension: publish a tool mapping

An administrator can map a new registry name to an existing built-in implementation or to a tool discovered on a registered MCP server. Capability and entity labels make the operation discoverable. A graph can reference that name and provide arguments matching the registered schema.

Registration does not create a handler, upload executable code or automatically grant every role access. Enabled state, role capabilities, data policy, risk and required permissions still govern invocation. A tool description is metadata, not an instruction that can override those checks.

Engineering extension: implement a new operation

A new first-party operation needs a typed input model, output model, handler, explicit risk/permission, timeout and registration in the backend. Tests should cover malformed arguments, authorization, tenant scope, side effects and uncertain outcomes. A remote REST API needs a reviewed native adapter or a compatible MCP wrapper before it can become a callable tool.

A meaningful extension contract

For a read-only sales-summary tool, define the account/business identifier, allowed date window, response metrics and missing-data semantics. For a write such as saving an internal investigation note, declare the write risk and pause for exact-argument approval. A future third-party mutation requires its own adapter, authorization and effect verification; declaring a capability label does not implement it.

The result is extensible capability discovery with a common enforcement path. It does not depend on allowing a model to invoke arbitrary functions.

Sources: [S03], [S05], [S06], [S07].

04 / GOVERNANCE AND GUARDRAILS

Extend policy without delegating authority to models

Governance is backend enforcement around resources, tools and data processing. Guardrails include deterministic authorization, contract validation, bounded execution and human decisions. The controls continue to operate even though the Governance screen is absent from the current navigation.

Runtime extension: configure existing policy

Authorized administrators can configure tool allowlists/denylists, maximum tool risk, tool-call and runtime ceilings, external-processing permission and approved MCP hosts through existing backend policy interfaces. Project access can restrict membership and mark data for local-only processing. Configuration must remain inside validated policy bounds.

Risk classes are read, internal_write, external_write and critical. A non-read tool requires an authorized approval bound to its exact arguments and current specification. The graph binding also includes the execution step; changing relevant tool/server configuration invalidates the old grant.

Engineering extension: add a new rule

Additional domain rules belong in reviewed deterministic code at the policy/adapter boundary, with explicit schema and tests. For example, a future spend-limited action could require a validated amount, account scope, ceiling check and provider receipt. That spend policy is an extension example, not a feature claimed implemented today.

There is no verified arbitrary policy-plugin engine or self-service policy scripting environment in this implementation. New risk classes or new decision attributes require coordinated changes across validation, storage, evaluation and tests.

Controls that every extension inherits

- Current organization membership and linked project/resource checks.
- Agent capability restrictions and tool argument validation.
- Data-processing restrictions for models and external MCP connections.
- Credential resolution outside prompts and source artifacts.
- Run/node budgets, cancellation, durable checkpoints and stale-worker fencing.
- One-time approval consumption and explicit handling of uncertain remote writes.

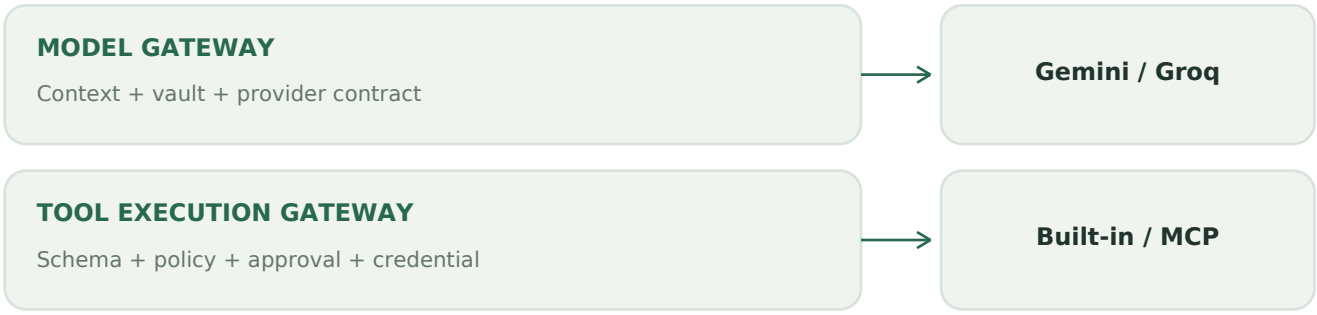
Prompts reinforce safe interpretation, while backend decisions enforce authority. Neither schema validation nor model self-review proves that an external integration is safe in every business situation.

Sources: [S01], [S02], [S03], [S05], [S06], [S12].

05 / GATEWAYS

Extend adapters at two distinct boundaries

Evolvyn has a model-access boundary and a tool-execution boundary. Both are application-level components. The inspected code does not establish a separately deployed, general-purpose network MCP reverse proxy.



Model gateway: interpretation and proposals

The agent/model layer resolves workspace/role context, obtains the matching vault credential, applies data policy, bounds input/output and calls a supported provider. Current user-facing AI setup supports Gemini and Groq. Model catalog checks and a JSON probe establish compatibility at the time of setup; later availability and quota can still fail.

Provider extensibility is an engineering capability. Adding a provider requires a fixed trusted destination, scoped credential support, model discovery/compatibility checks, normalized response/usage/error handling and relevant UI/API schemas. Tests must cover invalid output, quota/authentication failure, key revocation and data-policy denial. The PDF does not claim that arbitrary providers are currently selectable.

Tool execution gateway: validated operations

The execution path resolves a registered operation, validates its input, evaluates policy/approval, resolves any server-side credential, invokes the built-in/MCP adapter and validates the result as supported by its contract. Traces and relevant receipts are retained by the calling workflow or approval record.

Adapter extensibility means adding a reviewed implementation behind this path. A new transport or adapter type requires schema and dispatch changes plus tests; a registration alone cannot introduce an unsupported protocol. External MCP descriptions and results remain untrusted data.

Keeping the boundaries distinct prevents a model API response, a successful OAuth login or a healthy network session from being mistaken for business execution permission.

Sources: [S03], [S04], [S05], [S06], [S07], [S08], [S09], [S10].

06 / EXTENSIBILITY MAP

What can change, and how

COMPONENT	RUNTIME EXTENSION	ENGINEERING EXTENSION	BOUNDARY TO PRESERVE
MCP registry	Register approved HTTPS endpoints; probe and discover.	Add a reviewed profile, transport or provider authorization flow.	Host safety, credential scope, schema and timeout checks.
Tool registry	Map existing built-in/discovered tools; label capabilities and entities.	Implement typed handlers or new adapter dispatch.	Current permissions, declared risk, validation and approval.
Governance / guardrails	Configure allowed tools, ceilings, external processing and project access.	Add deterministic domain checks and coordinated schemas/tests.	Model output cannot grant authority or change policy.
Gateways	Use verified Gemini/Groq workspace keys and supported adapters.	Implement provider contracts, error/usage handling or new protocols.	Fixed destinations, scoped secrets, policy and output limits.
Harness / agents	Compose validated graphs and registered role/tool definitions.	Introduce new node semantics or recovery behavior through code.	Budgeted execution, checkpoints, state and stale-worker controls.

Shared acceptance rule

Every extension should preserve organization/project scope, secrets isolation, typed contracts, bounded execution, truthful failure reporting and the appropriate human decision boundary. New integrations must provide evidence of actual authorization and successful invocation before being represented as connected.

Configuration can extend the permitted vocabulary; code-level extensions can add new behavior. Neither route should create a parallel execution engine or an unrestricted side channel around policy.

Sources: [S01]–[S12].

07 / WORKED INTEGRATION EXAMPLE

Add a business capability through the same harness

Consider the implemented standalone MCP tool calculate_conversion. It takes supplied visits and paid counts and calculates a conversion rate. This example demonstrates extensible registration and invocation without claiming a live sales/account integration.

1. Deploy the authenticated MCP service with a private bearer token and a public HTTPS endpoint.
2. Save that same token as an MCP connection credential in the organization vault. Copy its credential reference, not the raw value, into registration.
3. Approve the hostname, register the endpoint and run health discovery. An unsuccessful probe remains unavailable.
4. Register the discovered tool under a stable platform name with suitable capability/entity labels and an honest read risk.
5. Add it to a permitted role or validated workflow graph. Supply arguments through the normal schema-checked path.
6. Invoke with visits=200 and paid=30. The deterministic result is 15%; the calling workflow can retain it as evidence.

Extending the same pattern to Growth Insights

A social observation tool needs a richer contract: provider/account identity, capture time, coverage, sampled post data and missing metrics. Evolvyn's social-source path requires a registered read-only MCP tool, binds its specification, validates observation identity/freshness and stores a snapshot for review.

The analyst interprets that snapshot; it cannot infer sales or unique customers from likes and comments alone. Monitoring can enqueue repeated collection through persisted schedules, but tokens, permissions, server availability and observation freshness still require operational checks.

If the extension writes data

A future external mutation must be classified as a write and use the approval path. The harness persists the proposed exact action, checks the current configuration when approval arrives, and handles failure or uncertainty without assuming the effect occurred. The deployed adapter must implement that operation; a model recommendation is not an implementation.

Sources: [S01], [S03], [S05], [S10], [S11], [S13].

08 / EVIDENCE AND PRODUCTION SCOPE

Extension points are supported by contracts

This brief is based on the inspected implementation at source revision 8ac1e56. It describes current registration/configuration surfaces and explicitly labels engineering extensions. It does not certify live third-party OAuth, unrestricted tool safety or production readiness.

Recorded verification

The earlier harness assessment ran the following focused suites on the same backend implementation: graph harness (7 tests), worker recovery (4), registry/governance (9), restricted-project policy (1) and model source generation (20). Result: **41 passed**, with one dependency deprecation warning. This document carries that result forward; it does not present it as a new test run.

Those tests exercise approval binding/replay protection, branches and joins, bounded loops/retries, stale-worker fencing, tenant/credential separation, unsafe MCP destinations and source-generation failure handling. Controlled provider boundaries do not prove successful authorization with a user's deployed account.

What to demonstrate to an evaluator

- Register a compatible MCP server, show actual discovery, and bind a usable tool.
- Invoke a read tool with schema-valid arguments and inspect its real result.
- Show rejection of malformed arguments, an unauthorized resource and an unsafe destination.
- Pause a write for approval, then demonstrate that stale/consumed authorization cannot be reused.
- Inspect a persisted execution/checkpoint and a provider failure state.
- Show the difference between configurable policy, a registered capability and a code-level adapter extension.

Next acceptance gates

Live provider/OAuth verification, backup/restore and vault-key recovery, workload monitoring, adversarial task evaluation and independent security review remain important. General policy scripting, generic provider onboarding and arbitrary generated-app execution should be treated as additional engineering work, not inferred from these extension points.

Sources: [T01], [T02], [T03], [T04], [T05].

09 / IMPLEMENTATION REFERENCES

Trace the architecture to source

All links target the inspected revision. The original Harness Engineering report remains a separate document; this brief provides dedicated coverage of extensible registries, guardrails and gateways.

REF	IMPLEMENTATION / EVIDENCE	REPOSITORY FILE
S01	Bounded graphs, approvals and checkpoints	backend/graph_harness.py
S02	Queue admission and worker leases	backend/execution_queue.py
S03	Resource, risk and data-processing policy	backend/policy.py
S04	MCP registration, discovery and remote sessions	backend/mcp_registry.py
S05	Tool mappings, invocation and approval binding	backend/tool_plugins.py
S06	Typed built-in implementation contract	backend/registry.py
S07	Agent definitions and bounded model access	backend/agent_gateway.py
S08	Scoped model routing and credential resolution	backend/model_access.py
S09	Provider catalog and JSON compatibility checks	backend/cloud_models.py
S10	Bundled MCP client execution	backend/mcp_gateway.py
S11	Authenticated standalone MCP server	backend/custom_mcp.py
S12	Validated policy configuration interfaces	backend/policy_api.py
S13	Source-bound social observation workflow	backend/social_growth.py
T01	Graph branches, approval and budget tests	tests/test_graph_harness.py
T02	Recovery, fencing and attempt-limit tests	tests/test_recovery.py
T03	Registry, approval and transport-boundary tests	tests/test_registry_governance.py
T04	Restricted project isolation test	tests/test_project_policy.py
T05	Source-contract and artifact persistence tests	tests/test_model_source.py

Key distinction for technical review

MCP registry: connection and discovery. **Tool registry:** callable operation and contract. **Governance:** permission and data-processing decisions. **Gateways:** controlled provider/adapter access. **Harness:** bounded orchestration, state, approvals and recovery.

These components are extensible within their supported interfaces. New protocols, providers, handlers and policy semantics require reviewed implementation and validation before deployment.